Reinforcement Learning

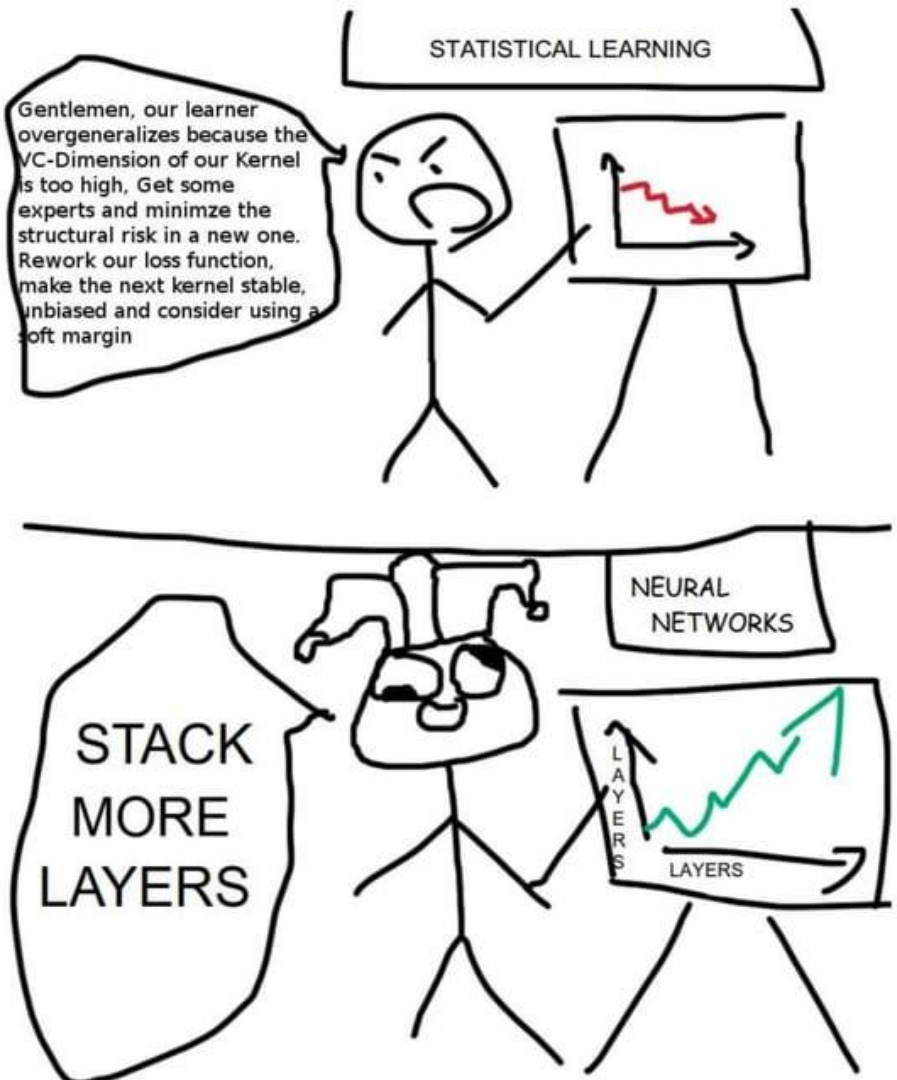
An Introduction

Course Content

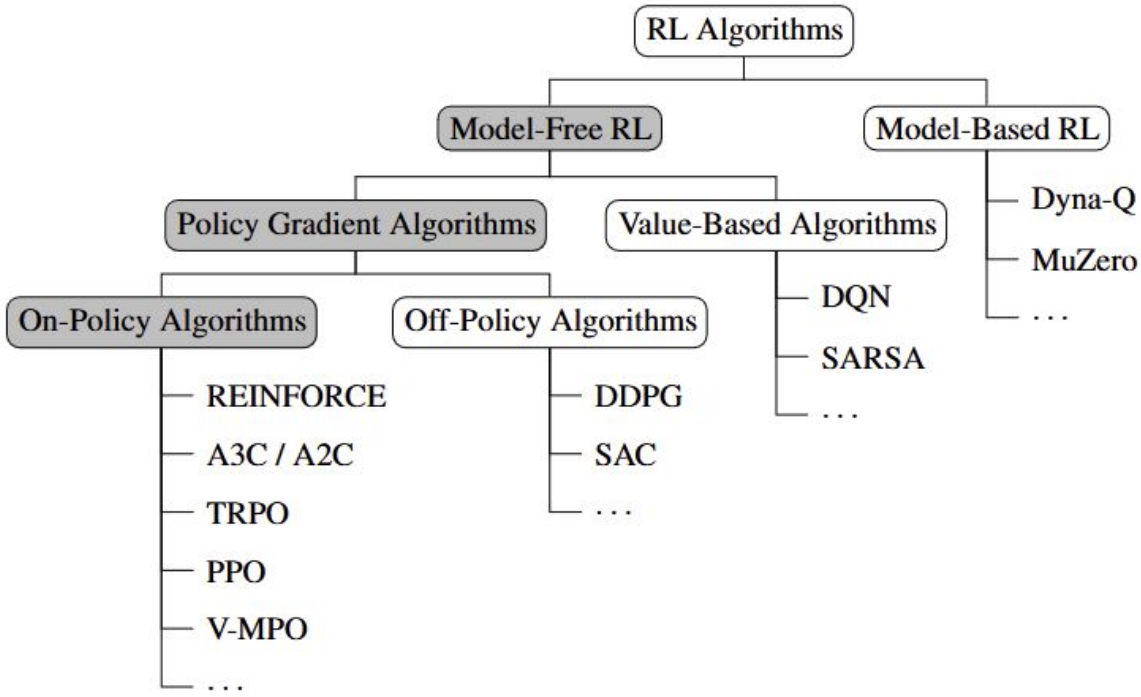
1. Introduction to Reinforcement Learning
2. Markov Decision Processes (MDPs)
3. Policy and Value Functions
4. Dynamic Programming (DP) for RL
5. Model-Free methods
6. Value Function Approximation
7. Policy-Gradient and Actor-Critic Methods
8. **Deep RL**
9. TP Project

Deep RL Methods

Deep RL Methods

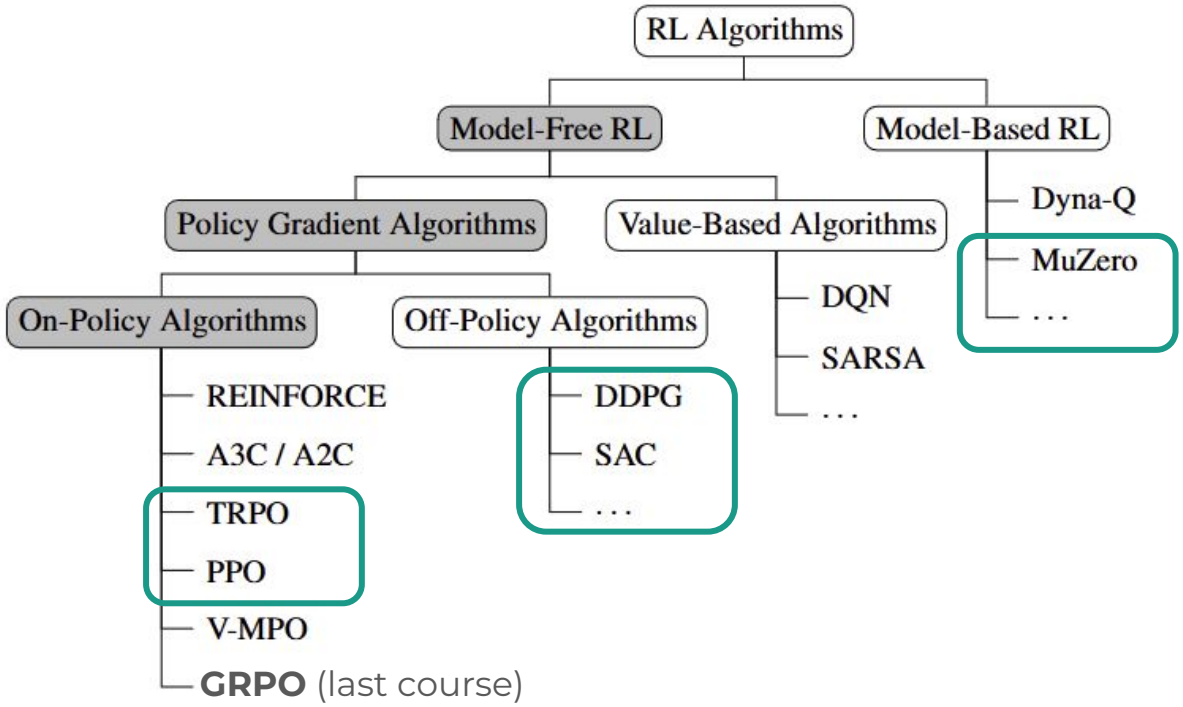


Summary



Source

Summary - Today



Source

Off-Policy

Twin Delayed Deep Deterministic Policy Gradient - TD3

- **Actor Critic, Off-policy**, **model-free** reinforcement learning algorithm for continuous action spaces
- Improves the Deep Deterministic Policy Gradient (**DDPG**) algorithm
- Implements three key innovations to enhance stability and performance:
  1. Clipped Double Q-learning to address overestimation bias
  2. Delayed policy updates
  3. Target policy smoothing
- Utilizes a **replay buffer** for improved sample efficiency

Algorithm 1 TD3

1 Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ
Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$

2 Initialize replay buffer $\mathcal{B}$

for $t = 1$ **to** T **do**

3 Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'

4 Store transition tuple (s, a, r, s') in $\mathcal{B}$

5 Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$
$\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \mathrm{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$
$y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$
Update critics $\theta_i \leftarrow \mathrm{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$

6 **if** t mod d **then**
Update ϕ by the deterministic policy gradient:
$\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$

7 Update target networks:
$\theta'_i \leftarrow \tau\theta_i + (1 - \tau)\theta'_i$
$\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$

end if

end for

SAC - Soft Actor Critic

- **Off-policy**, model-free reinforcement learning algorithm for continuous action spaces
- Combines **actor-critic** architecture with maximum entropy reinforcement learning
- *Two critic networks* to reduce overestimation bias
- Well-suited for high-dimensional, continuous control tasks like robotics
- Generally more sample-efficient than on-policy methods like **PPO**
- Achieves state-of-the-art performance on many **continuous control** benchmarks

On-Policy

TRPO

Trust-Region Policy Optimization

- An Actor-Critic Algorithm
- Employs natural **policy gradient**
- Uses **KL-divergence** constraint to limit policy updates, ensuring stability
- Guarantees monotonic policy improvement by optimizing within a **"trust region"**

PPO - Combine everything !

- An Actor-Critic Algorithm
- Using **Gradient Clipping**
- Combines ideas from **A2C** ...
  - Advantage
  - Using an entropy bonus for exploration
- ... and **TRPO**
  - Trust region to improve stability and avoid catastrophic drops in performance

PPO is an on-policy algorithm, which means that the trajectories used to update the networks must be collected using the latest policy. It is usually less sample efficient than off-policy algorithms like DQN, SAC or TD3, but is much faster regarding wall-clock time.

Model - Based RL

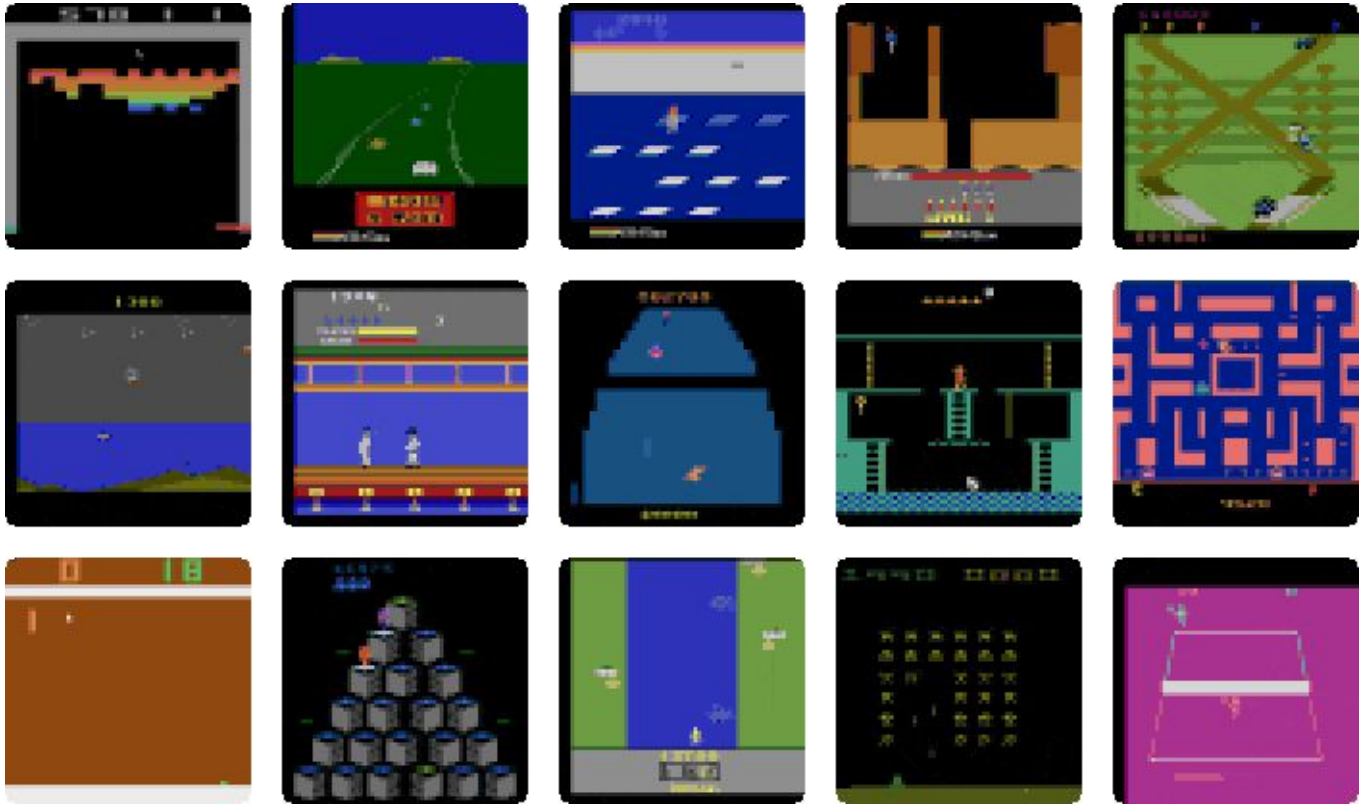
Principle of Model-Based RL

- Model-based agents are attractive for several reasons.
- They let you **plan from among several imagined futures**,
  seems like something an actually intelligent agent would be able to do right ?
- in fact they're pretty hard to put together well.

Now let’s explore the path of a very well performing model, and how research was lead to find it

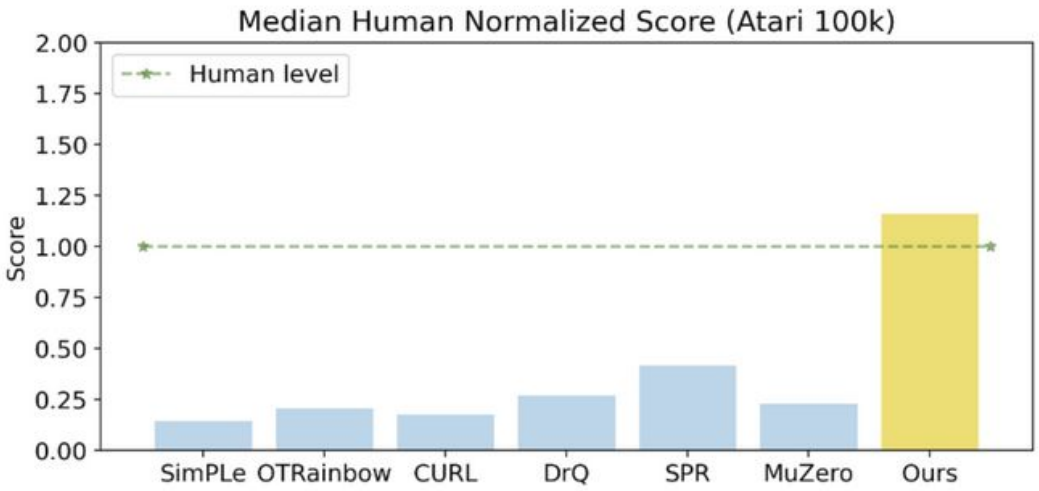
From AlphaGo to EfficientZero

Know about Atari Games ?



EfficientZero BreakTrough

Mastering Atari Games with Limited Data, Ye et al. NeurIPS 2021



- From Academia !
- **116% of human median performance on the data-limited Atari 100k benchmark.**
- Previously-best algorithm only reached about 41% of median human performance

How is that possible ??

The Game of Go

- After success of **Deep Blue** against Kasparov (1996), *Go was the last board game standing in front of AI*..
- Game tree complexity ? $\boxed{N \approx 10^{170}}$
- **“Impossible” for computers to evaluate who is winning**
- Until then we only coded algorithms exploring the whole game, but here it is too big !

AlphaGo

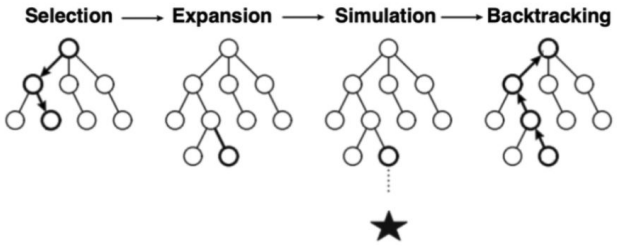
An Imitation baseline

- Developed by DeepMind
- Uses Monte Carlo Tree Search (MCTS)
- Defeated Lee Sedol (world Champion of Go) in march 2016
- Trained by *imitation learning and Self Play*

Monte Carlo Tree-Search : MCTS

Starting from a tree restricted to the root, repeat N times:

- **Selection**: Select a leaf starting from the root using some opportunistic algorithm (e.g.,ε-greedy)
- **Expansion**: Expand this leaf with all possible actions and select one of these actions at random → state s
- **Simulation**: Estimate the outcome of the game from state s using some default policy (e.g., random)
- **Back-propagation**: Update the action value function on the path from state s to the root
  Select the best action according to the value of each action



💡 Intuitive: explore several possible branches of the upcoming episode
💵 *Computationally expensive..*

Better - AlphaZero

Self supervised version

- One year later by DeepMind (Oct 2027, Nature)
- **Remove the dependency to human knowledge !**
- Still model-based, rules are hard-coded

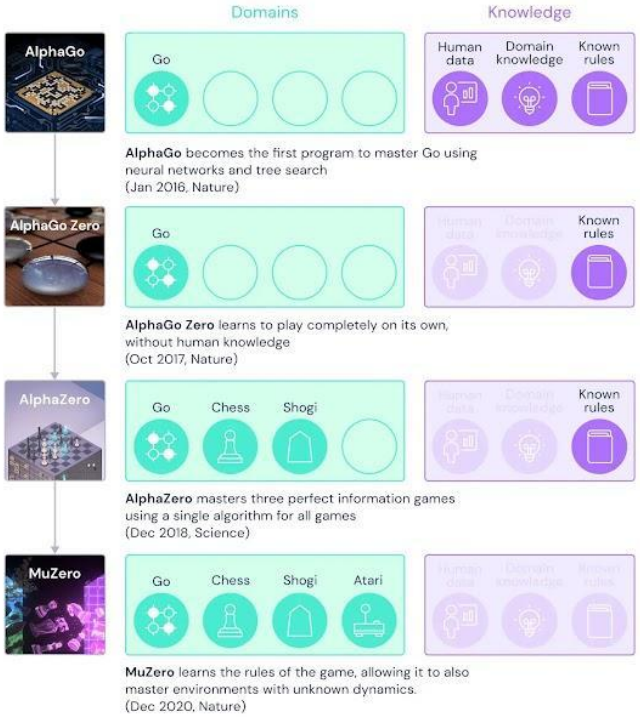
Both AlphaGo and AlphaZero, however, were limited in one particular and important way: they had to possess hard-coded models of the game.

Even Better - MuZero

- Still DeepMind...
- **No More hard-coded Rules**
- Still model-based

Models three elements of the environment that are critical to planning:

- The **value**: how good is the current position?
- The **policy**: which action is the best to take?
- The **reward**: how good was the last action?
    - This makes it **Model-Based**



Even Better - MuZero

Usage of MCTS to better forecast the Game

[5] - DeepMind MuZero Post

Finally - EfficientZero

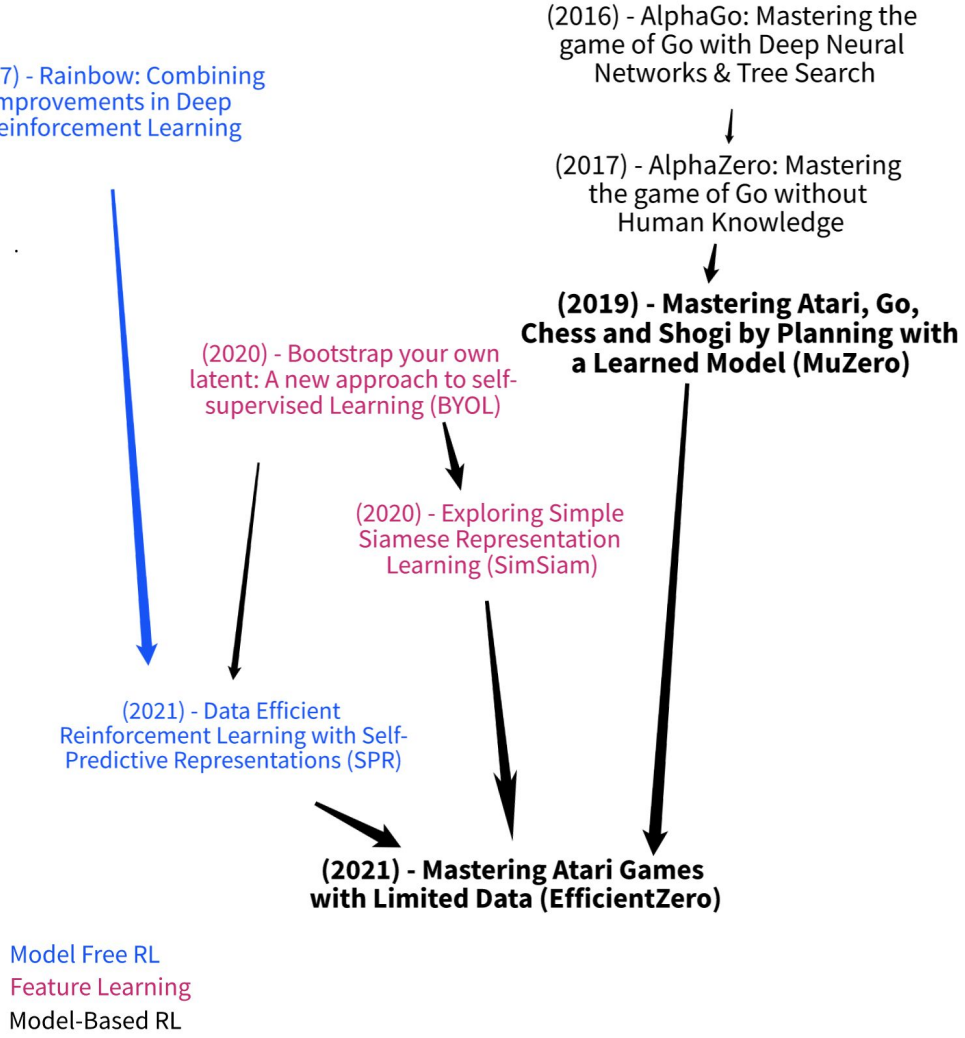
Still used MCTS the same way as MuZero, but adds a few more tricks

- Self supervised consistency Loss.
- Fixes the sparse reward problem for the prediction by comparing the prediction vs next step for each timestep !
- Value Prefix
  - replace the reward by the sum of predicted rewards for a given time window.
- Off-Policy correction.

[4] - Efficient Zero Medium Post

EfficientZero - rollBack

As for many SoTA algos in RL and in AI in general, the best performing models often merge many tips and tricks from previous work.



Any Questions ?

Contact us !

morand@isir.pmc.fr

References

[1] David Silver, Lectures on Reinforcement Learning, 2015

[2] Reinforcement Learning and Advanced Deep Learning (Sorbonne) - Olivier Sigaud

[3] Sutton, R. S. and Barto, A. G. (2018), Reinforcement Learning: An Introduction (Second edition). MIT Press

Olivier Sigaud Youtube Channel

[4] - Efficient Zero Medium Post